- command-r : Performs language tasks at a higher quality, more reliably & with a longer context than previous models.

- command-nightly & command-light-nightly : The latest, most experimental, and (possibly) unstable versions of their default counterparts. Not recommended for production use.

↳ Use the model parameter to select a variation that suits your requirements. In the code cell, we select command-r.

```
response = co.chat (message = "Hello",
                    model = "command-r-plus")
print (response.text)
```

O/P

```
# Response
Hello! How can I help you today?
```

↳ Randomness

Often you'll need to control the level of randomness of the model. There are a no. of factors to keep in mind when tuning model randomness, including:

- command-r : Performs language tasks at a higher quality, more reliably & with a longer context than previous models.

- command-nightly & command-light-nightly : The latest, most experimental, and (possibly) unstable versions of their default counterparts. Not recommended for production use.

↳ Use the model parameter to select a variation that suits your requirements. In the code cell, we select command-r.

```
response = co.chat (message = "Hello",
                    model = "command-r-plus")
print (response.text)
```

O/P

```
## Response
Hello! How can I help you today?
```

↳ Randomness

Often you'll need to control the level of randomness of the model. There are a no. of factors to keep in mind when tuning model randomness, including:

- Task type: You'll likely want to decrease the randomness of the model when performing structured tasks that have a correct answer, like for question answering or summarization or for generating technical text. In these cases, we want the model to yield a safe & predictable response. On the other hand, if you are generating poetry or brainstorming ideas, you might want to increase the randomness to produce more diverse & creative responses

- Model behavior: We likely need to increase the randomness of the model if it gets stuck in a loop & starts repeating itself, or if it is producing overly generic phrases. Increasing the randomness will expand the set of words that the model can use when generating responses.

- Controlling style & tone: If generating text that needs to have a specific tone or style like for a company blog post or customer support responses, a low level of randomness might be desirable, to keep the model from generating unusual words.

↳ Modifying the temperature parameter changes the extent to which the model considers incorporating unlikely tokens (can be words, parts of words, or punctuation) in its response, which can make the O/P more random & creative.

To understand th
The model would
cookies has a
for appearing

I like to bo

The model
of all po

Before these lik
they first need to
parameter contro

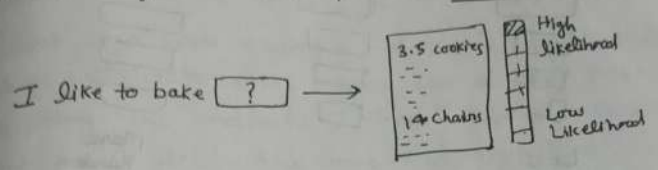
- At low te
very low p
a high

- At high
look
high li
higher

randomness
that have
summarization
es, we
response.
try or
ease the
creative

the
loop
overly
l
use
;
hat
a
nses,
ole,
ords,
e
ly

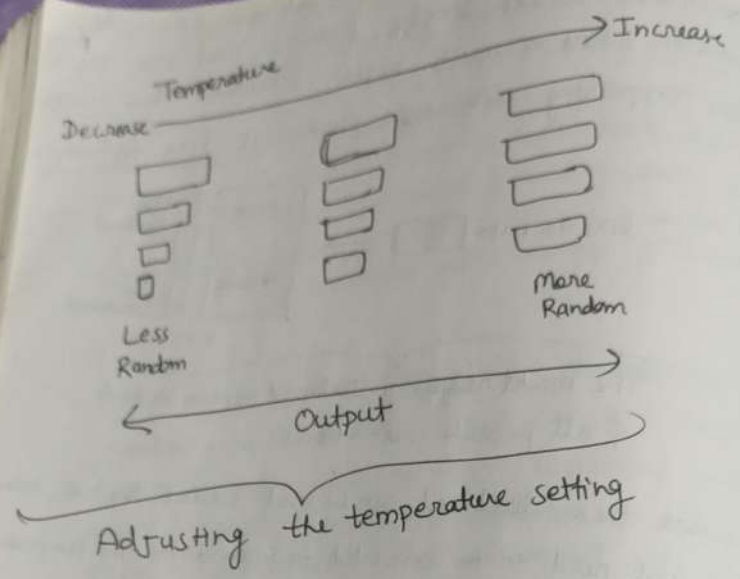
To understand this, we'll look at an example. The model would likely predict that the token cookies has a much higher likelihood than chairs for appearing after the phrase I like to bake--



The model assigns a likelihood number to each of all possible next tokens.

Before these likelihoods can be used to select the next token, they first need to be converted to probabilities. The temperature parameter controls how this conversion is done.

- At low temperature, low likelihood tokens are assigned very low probabilities & high " " " " " high " .
- At high temperature, the probabilities will look roughly similar for each token, with high likelihood tokens assigned only slightly higher probability.



Building off the example above,

- At low temperature, there's a probability that chair is selected, but the prob. is slightly lower than cookies.
- At high temperature, the prob. that chair is selected is only slightly lower than Cookies.

The temperature parameter is a value b/w 0 & 1. As you increase the temperature, the model gets more creative & random. Temperature can be tuned for different prob^{m}s, & most people will find that the default temperature of 0.3 is a good starting point.

Let's look at a code example, where we suggest that the model generate alternative names for a blog post. Prompting the endpoint 5 times when the temperature is set to 0 yields the same o/p each time.

```
message = """ Suggest a more exciting title for a
        blog post titled: Intro to Retrieval-Augmented
        Generation.\ Respond in a single line."""

for _ in range(5):
    response = co.chat(message = message,
                       temperature = 0,
                       model = "command-r-plus")

    print(response.text)
```

O/p.

```
#Response
The Future of AI: Unlocking the Power of Retrieval Augmented Generation
```

However, if we increase the temperature to the max^m value of 1, the model gives diff proposals.

[Same code with temperature = 1)

O/P

Response.

Unleashing the Power of Retrieval-Augmented Generation

The Exciting future of AI! - -

The magic of AI! - -

The Future of AI! - -

" Unleasing the Power of AI! The Rise of - -)

⇒ Prompt Engineering Basics

Here, you'll learn basics of prompt engineering & how to craft effective prompts to obtain desirable O/P's for various tasks.

→ Prompting is at the heart of working with LLMs.. The prompt provides context for the text we want the model to generate. The prompts we create can be anything from simple instruct^ns to more complex pieces of text, & they are used to encourage the model to produce a specific type of output.

→ Coming up with a good prompt is a bit of both science & art.

↳ Setup.

```
import cohere
co = cohere.Client("Cohere-Api-key").
```

Let's also define a function generate_text() to take a given message, call the Chat endpoint & stream the response.

```
def generate_text (message):
    stream = co.chat_stream(message=message,
                            model = "command-r-plus")
    for event in stream:
        if event.event_type == "text-generation":
            print (event.text, end='')
```

Writing a Basic Prompt

The best way to design prompts for a model like Command is to give a command or an instruction. One way to do this is by using imperative verbs, for example: generate, write, list, provide & other variations.

For instance, let's say that we are creating the product description copy for a wireless earbuds product. We can write the prompt as follows:

```
generate_text ("Generate a concise product description
                for the product: wireless earbuds.")
```

O/P # Response

Sure! Here is a concise product description for wireless earbuds:
"Experience unparalleled sound with our wireless earbuds. Immerse yourself in rich, high quality audio & enjoy

That's not bad. With a simple, one-line prompt, we already have a piece of product description that can make a digital marketer proud.

↳ Layering Additional Instructions.

But perhaps we want to be more specific regarding what we want the output to look like. For this, we can layer additional instructions onto the model in the prompt.

Let's say we want the model to write the product description in a particular format with specific info^n. In this case, we can append this specific instruction in the prompt.

```
generate_text("""
    Generate a concise product description for the product:
wireless earbuds.
Use the following format: Hook, Solution, Features. and
Benefits, Call to Action.
    """)
```

O/P #Response

Here is a concise product description for wireless earbuds:

** Hook:** Tired of tangled cords and restricted movement?

** Solution:** Go wireless with our latest earbuds!

** Features & Benefit:**

- Sleek & ergonomic - —

** Call to Action:** Experience true - —

The model returns an output following the format that we intended.

Adding Context to a Prompt

The prompt can also be constructed as a combination of an instruction & some context. Let's see this in action with another example: emails.
We can create a prompt to summarize an email, which is included in the prompt for context.

```
generate_text(""
    Summarize this email in one sentence.
    Dear [Team Members],
    I am writing to thank you - -
    - - - -
    - - -
    """)
```

O/P # Response

The email expresses gratitude to the team members for their hard work. —

→ This instruction-context prompt format is extremely useful as it means that we can supply additional info^n as context to help ground the model's output. One such example is a question-answering system for, let's say, a company's knowledge base. Given a question (the instruction), the model will only be able to provide accurate answers if provided with the knowledge base (the context).